

APTC106

Semana 6 – Sumativa 2

Propuesta de aplicación

Integrantes: Joaquin Olivares, Javiera Chávez, Claudio Yáñez.

Asignatura: Desarrollo web y Movil.

Docente: Ernesto Vivanco Tapia.

Fecha: 04/08/2026.

Índice

1. Introducción.....	3
2. Adaptación del código base.....	4
2.1 Roles y flujo de negocio implementado	4
2.2 Cambio de stack tecnológico respecto a la propuesta original	5
3. Análisis de funcionamiento	5
3.1 Hallazgos técnicos durante las pruebas	6
4. Análisis crítico	6
4.1 Uso del código base entregado	6
4.2 Modelo de roles mediante UserProfile	7
4.3 Modelo de estado como campo de texto	7
4.4 Alcance de las pruebas realizadas	7
5. Conclusiones	8

1. Introducción

En la sumativa 1 se propuso una solución de logística inversa para el comercio electrónico, orientada a digitalizar el proceso de devoluciones y cambios de productos mediante la integración de una plataforma web y una aplicación móvil. Dicha propuesta identificó la logística inversa como un eslabón débil y poco automatizado del comercio electrónico -consistente con lo señalado por Corral Cáceres (2022) sobre la gestión manual y poco estandarizada de las devoluciones, y por Portal Innova (2025) respecto al comercio electrónico chileno-, y definió cinco roles (cliente, ejecutivo de postventa, ejecutivo de logística, operador de logística y administrador) que participa en el ciclo de vida de una solicitud de devolución.

La motivación de este informe es continuar ese trabajo llevándolo del plano conceptual a una primera implementación funcional. Para ello, se tomó como base un proyecto CRUD y se adaptó al dominio de logística inversa definido en la propuesta original, reemplazando su modelo de dato su su lógica de negocio por los correspondientes a la gestión de solicitudes de devolución.

El objetivo general de este informe es documentar y analizar críticamente la adaptación de dicho CRUD, evidenciando que su funcionamiento mínimo viable responde a los roles y al flujo de proceso definidos en la propuesta de solución.

Los objetivos específicos son:

- Describir cómo se adaptó el código base entregado al caso de logística inversa.
- Presentar el análisis de funcionamiento del sistema, evidenciando su despliegue y ejecución en un entorno local.
- Argumentar críticamente las decisiones de diseño tomadas durante la adaptación.
- Concluir sobre el nivel de logro de los objetivos y definir un marco de acción futura.

2. Adaptación del código base

El punto de partida fue el proyecto simple-crud-django, un CRUD en Django con vistas basadas en funciones y autenticación, originalmente contraído sobre una entidad de películas (Movies/Categories). Sobre esa base se incorporaron dos modelos propios del dominio de logística inversa: UserProfile, que asocia a cada usuario un rol dentro del proceso y SolicitudDevolucion, que representa el objetivo central del negocio (producto, motivo, evidencia fotográfica, estado y fecha de creación).

La entidad original (Movies/Categories) se conservó en el proyecto como parte de la base heredada, pero no forma parte del flujo de negocio implementado; toda la lógica de logística inversa se construyó sobre SolicitudDevolucion y UserProfile.

2.1 Roles y flujo de negocio implementado

Se implementó control de acceso por rol en las vistas, replicando la tabla de actores definida en la propuesta original:

Rol	Acción principal en el sistema
Cliente	Crea una solicitud de devolución/cambio adjuntando evidencia y consulta el estado de sus solicitudes.
Ejecutivo Postventa	Revisa las solicitudes, valida la evidencia y aprueba o rechaza cada solicitud.
Ejecutivo de Logística	Coordina el retiro y genera la etiqueta de reenvío de las solicitudes aprobadas.
Operador de Logística	Ejecuta el retiro/envío una vez generada la etiqueta, cerrando la solicitud.
Administrador	Accede a todos los paneles, visualiza indicadores y descarga el reporte en CSV.

El estado de una solicitud avanza según la siguiente secuencia, reflejada en el campo estado del modelo SolicitudDevolucion y en las vistas de acción correspondientes:

Pendiente → Aprobada → Retiro Coordinado → Etiqueta Generada → Completado (o Rechazada, si el ejecutivo de postventa rechaza la solicitud)

2.2 Cambio de stack tecnológico respecto a la propuesta original

La propuesta planteaba como alternativa un stack basado en JavaScript (React para la plataforma web, React Native + Expo para la aplicación de móvil, Node.js + Express para backend y MongoDB como base de datos), fundamentado en la ventaja de compartir un mismo lenguaje entre frontend y backend.

Para esta etapa se optó por implementar el CRUD sobre Django, ya que corresponde al código base entregado como un punto de partida obligatorio. Esta decisión implica un desvío respecto del stack propuesto originalmente, que se documenta y justifica en la sección de análisis crítico de este informe.

3. Análisis de funcionamiento

El sistema fue desplegado localmente y sometido a una prueba de extremo a extremo, ejecutando el flujo completo de negocio con un usuario real por cada rol definido en el sistema. La siguiente tabla resume las pruebas realizadas y sus resultados:

Prueba realizada	Resultado
Instalación de dependencias y ejecución de manage.py check	Sin errores (1 advertencia menor no bloqueante sobre la carpeta static).
Aplicación de migraciones (manage.py migrate)	Ejecutada correctamente sobre la base de datos del proyecto.
Inicio de sesión con un usuario por cada rol (cliente, postventa, ejecutivo de logística, operador de logística, administrador)	Cada usuario fue redirigido a su panel correspondiente según su rol.
Creación de una solicitud de devolución como cliente, incluyendo evidencia adjunta	La solicitud quedó registrada en la base de datos con estado Pendiente.
Aprobación de la solicitud por el ejecutivo de postventa	El estado cambió correctamente a Aprobada.
Coordinación de retiro y generación de etiqueta por el ejecutivo de logística	El estado avanzó a Retiro Coordinado y luego a Etiqueta Generada.
Ejecución del retiro/envío por el operador de logística	El estado final quedó registrado como Completado.

Acceso al panel de administrador, indicadores y descarga de reporte CSV	Los indicadores reflejaron correctamente el total de solicitudes, completadas y pendientes; la descarga del CSV se generó sin errores.
---	--

3.1 Hallazgos técnicos durante las pruebas

Durante el análisis de funcionamiento se identificaron tres observaciones relevantes, que se abordan en el análisis crítico y quedan documentadas como trabajo futuro:

- Las visitas que ejecutan las transiciones de estado (`coordinar_retiro_accion`, `generar_etiqueta_accion` y `ejecutar_retiros_envios_accion`) no validan el estado previo de la solicitud, por lo que en la versión actual es técnicamente posible saltarse pasos del flujo.
- La verificación de rol de la vista `ejecutar_retiros_envios_accion`, quedó comentada en el código durante una etapa de pruebas y no fue reactivada, por lo que cualquier usuario autenticado puede actualmente marcar una solicitud como Completado.
- El proyecto no incluye una operación de eliminación (Delete) para la entidad `SolicitudDevolucion`; el rechazo de una solicitud cumple una función equivalente dentro del flujo de negocio, pero no constituye una eliminación del registro.

Adicionalmente, se corrigió la documentación de instalación de instalación del proyecto (README y archivo de dependencias), que originalmente referenciaba una versión de Django incompatible con el entorno de ejecución utilizado (Python 3.12) y no incluía la dependencia `django_unfold`, requerida por la configuración del proyecto. Esta corrección fue necesaria para garantizar que el despliegue local descrito pueda reproducirse siguiendo únicamente las instrucciones documentadas.

4. Análisis crítico

4.1 Uso del código base entregado

Se decidió partir del CRUD entregado en lugar de construir el backend desde cero con el stack JavaScript propuesto con anterioridad. Esta decisión prioriza el objetivo específico de esta actividad – adaptar y demostrar el funcionamiento de un CRUD concreto– por sobre la coherencia total con la arquitectura conceptual previamente propuesta. El costo de esta decisión es que la implementación actual no es directamente compatible con una futura aplicación móvil en React Native tal como se planteó originalmente, ya que Django no expone de forma nativa una API REST; el

beneficio es contar con una base de autenticación, control de acceso y persistencia de datos ya probada, sobre la cual fue posible construir el modelo de negocio en el tiempo disponible.

4.2 Modelo de roles mediante UserProfile

Para representar los cinco roles del proceso se optó por un modelo propio (UserProfile) asociado uno a uno con el modo de usuario de Django, en lugar de utilizar el sistema de grupos y permisos (Group/Permission) que Django ofrece de forma nativa. Esta decisión simplificó la implementación, ya que permitió centralizar la verificación de rol en una sola consulta (request_user_profile_role) y expresa las reglas de acceso directamente en cada vista. La contrapartida es que la validación de permisos quedó distribuida de forma manual en cada vista en lugar de apoyarse en el mecanismo de permisos de Django, lo que – como se documentó en la sección anterior – permitió que una verificación de rol quedara deshabilitada sin que el sistema lo advirtiera.

4.3 Modelo de estado como campo de texto

El estado de una SolicitudDevolucion se modeló como un campo de texto con opciones (choices), y cada vista de acción actualiza ese campo de forma directa. Esta aproximación es simple de implementar y de leer, y fue suficiente para demostrar el flujo completo de negocio en esta etapa. Sin embargo, al no existir una validación explícita de las transiciones permitidas, el modelo no impide que una solicitud avance de forma inconsistente (por ejemplo, coordinar un retiro sobre una solicitud que no ha sido aprobada). Se optó por no resolver esta limitación en esta entrega, priorizando dejar funcionando el flujo completo por sobre construir una máquina de estados más estricta, y se deja documentada como mejora futura.

4.4 Alcance de las pruebas realizadas

El análisis de funcionamiento de este informe se basa en pruebas manuales de flujo completo, ejecutadas con un usuario real por cada rol sobre un despliegue local. Se priorizó este tipo de prueba, cercana al uso real del sistema, por sobre la construcción de pruebas automatizadas (unitarias o de integración), dado el tiempo disponible para esta entrega. Esta decisión permitió validar rápidamente que el CRUD adaptado cumple su función mínima viable, pero no protege al proyecto de regresiones futuras, por lo que la incorporación de pruebas automatizadas queda planteada como parte del marco de acción futura.

5. Conclusiones

El objetivo de esta etapa era adaptar el CRUD entregado al caso de logística inversa propuesto anteriormente y demostrar su funcionamiento mínimo viable. Ese objetivo se cumplió: el sistema implementado permite ejecutar de principio a fin el flujo definido originalmente —un cliente crea una solicitud de devolución, un ejecutivo de postventa la revisa y aprueba, un ejecutivo de logística coordina el retiro y genera la etiqueta, y un operador de logística ejecuta el envío hasta cerrar la solicitud— con control de acceso diferenciado por rol para los cinco actores definidos en la propuesta original.

El análisis de funcionamiento realizado sobre un despliegue local confirmó que los componentes centrales del sistema —autenticación, persistencia de datos, carga de evidencia y generación de indicadores— operan correctamente. Al mismo tiempo, el proceso de pruebas permitió identificar de forma crítica tres limitaciones concretas de la implementación actual (transiciones de estado sin validar, una verificación de rol deshabilitada y la ausencia de una operación de eliminación explícita), lo que evidencia que el equipo no solo verificó que el sistema funcionara, sino que evaluó con criterio los riesgos de la solución construida.

Como marco de acción futura, se propone: (1) reactivar y reforzar las validaciones de rol y de estado previo en las vistas de transición identificadas en este informe; (2) evaluar la exposición de una API REST sobre el backend actual, de modo de acercar la implementación a la arquitectura de plataforma web más aplicación móvil planteada en la propuesta original; y (3) incorporar pruebas automatizadas que cubran el flujo completo de una solicitud de devolución, reduciendo la dependencia de pruebas manuales para futuras iteraciones del proyecto.